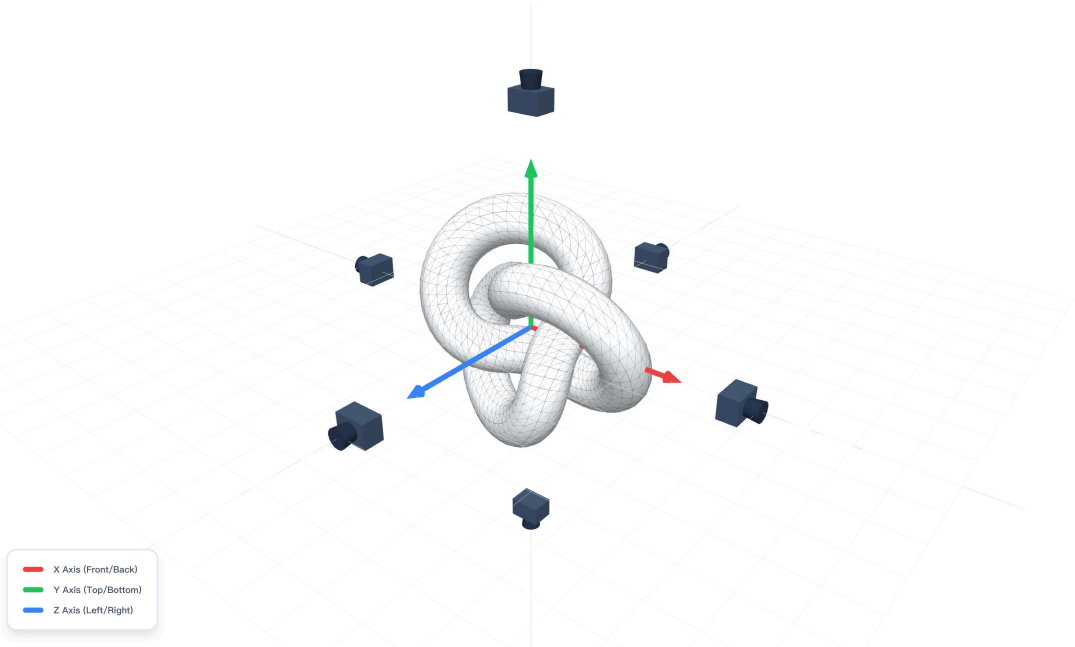
Problem A
Perception-Aware Lossless Simplification of Million-Vertex 3D Meshes for Mobile Platforms

Business Background & Competition Overview

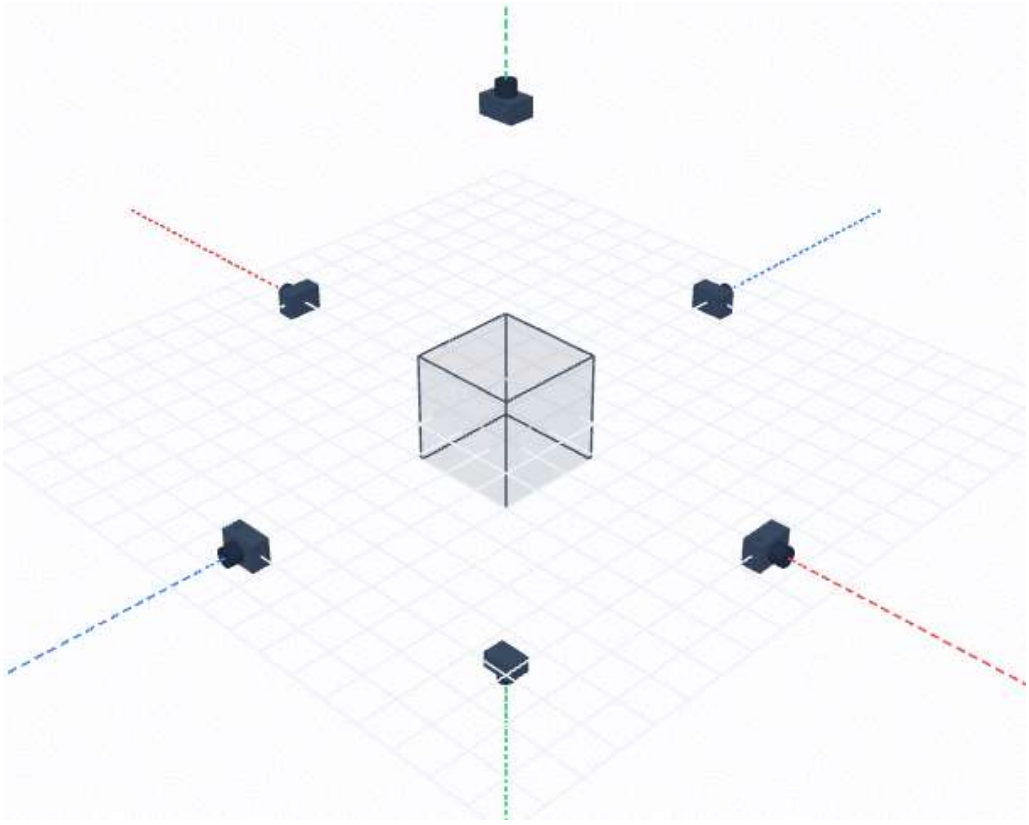
Balancing geometric complexity and visual fidelity is a core technical challenge for mobile 3D development, digital twin and other scenarios with strict real-time rendering performance requirements. In this competition, participants are required to compress high-complexity original high-poly meshes with millions of vertices as aggressively as possible – using as few vertices as they can – while keeping the result visually faithful to the original and maintaining complete basic topological structure. Different from traditional geometry-deviation-oriented algorithms, this competition emphasizes perception-driven optimization. Participants shall adopt perceptual metrics (e.g., pixel-level rendering consistency, visual saliency features) to ensure the simplified mesh achieves visually indistinguishable effects from the original high-poly model in light-shadow performance, contour edges and key details.



System Model: Standardized Virtual Photography Evaluator

To quantitatively measure the visual consistency between the simplified mesh M' and the original high-poly mesh M, the system integrates an offline

rendering pipeline. Centered on the mesh origin, the evaluator places virtual cameras along six positive/negative axial directions in the 3D Cartesian coordinate system to build a multi-view sampling space.



Camera Placement & View Indexing

For each view $k \in \{1, 2, 3, 4, 5, 6\}$, the camera viewpoint E is defined as:

$$E_k \in \{(\pm D, 0, 0), (0, \pm D, 0), (0, 0, \pm D)\}$$

- D: Fixed observation distance, $D = 2.5$ (model unit).
- $\pm X, \pm Y, \pm Z$: Cameras face the mesh origin from positive/negative directions of X, Y, Z axes respectively.

The input mesh is given pre-normalized (see the Input section): it is centered at the origin and scaled to lie within the unit sphere ($\|v\| \leq 1$). The evaluator renders these coordinates directly at distance D, with no further scaling or recentering, so the fixed camera always frames the model identically.

From each view, the system renders 3D geometric data into two types of feature images for subsequent similarity scoring.

Multi-dimensional Visual Feature Extraction

The system generates feature maps from light-shadow and geometric morphology dimensions to calculate SSIM (Structural Similarity Index Measure).

Normal Map: Capturing Light-shadow Undulation

Surface normal determines light reflection effect. The evaluator uses **flat shading**: each triangular face has a single unit normal, and every pixel covered by that face is assigned this same face normal (normals are *not* interpolated across the face – the input provides no per-vertex normals). The normal is mapped to RGB color values:

$$I_N(P) = (\boldsymbol{n}_p + [1,1,1]^T) \times 127.5$$

- $\boldsymbol{n}_p$: Unit normal of the triangular face covering pixel p (constant across that face), with three components ranging from $[-1, 1]$.
- $[1,1,1]$: Offset each normal component by $+1$ to normalize the range to $[0, 2]$.
- Multiply by 127.5: Remap values to $[0, 255]$ to generate standard RGB pixel values.

Depth Map: Capturing Contour Morphology

This map records the depth value z (distance along the camera's viewing axis) at each pixel. For any pixel p inside a triangular face, the depth is obtained by **perspective-correct** interpolation of the three vertex depth values (see Section 2):

$$I_D(p) = Interpolate(z_0, z_1, z_2)$$

- z_0, z_1, z_2: Camera-space Z-axis depth coordinates of three triangle vertices.
- $Interpolate$: **Perspective-correct** interpolation based on barycentric coordinates – the reciprocal $1/z$ is interpolated linearly in screen space (see Section 2).

Evaluation Purpose: Monitor mesh volume shrinkage and contour aliasing, ensuring consistent occlusion relationship and edge silhouette between simplified and original meshes.

Attribute Sampling: Mapping 3D Geometry to 2D Pixels

To generate the above feature maps, the system builds precise mapping from 3D space to 2D pixel arrays, including two core steps: spatial projection and attribute interpolation.

Perspective Projection Formula

The camera is placed at the origin facing the negative Z-axis (OpenGL standard). For any 3D vertex $V(x, y, z)$ of the mesh, the projected 2D screen pixel coordinate $p(u, v)$ is calculated as follows:

$$u = f_x \cdot \frac{x}{z} + c_u, v = f_y \cdot \frac{y}{z} + c_v$$

- (x, y, z): 3D coordinates under camera space (camera as origin, sight along negative Z-axis).
- f_x, f_y: Virtual camera focal length (pixel unit), fixed as $f_x = f_y = 800.0$.
- c_u, c_v: Image principal point offset, equal to half feature map width and height to center projected mesh.
- (u, v): Floating-point pixel index on feature map, rasterized to discrete grid pixels subsequently.

Attribute Interpolation: Barycentric Coordinate Method

After confirming which triangle contains pixel P, the system assigns its attributes. The **normal** is the covering face's flat normal (see the Normal Map

definition above), so it needs no interpolation. The **depth** is interpolated from the three vertex depths z_0, z_1, z_2 using barycentric coordinates as weights. For pixel P, weight coefficients (w_0, w_1, w_2) satisfy the equation below:

$$P = w_0 p_0 + w_1 p_1 + w_2 p_2$$

- p_0, p_1, p_2: 2D screen coordinates of three projected triangle vertices.
- w_0, w_1, w_2: Barycentric weights representing contribution of vertex v_0, v_1, v_2 to pixel P. And $w_0 + w_1 + w_2 = 1$.

Based on these barycentric weights, the per-pixel depth is interpolated **perspective-correctly** – the quantity that is linear in screen space is the reciprocal $1/z$, not z itself:

$$z_P = \frac{1}{\dfrac{w_0}{z_0} + \dfrac{w_1}{z_1} + \dfrac{w_2}{z_2}}$$

- z_{v_i}: Camera-space depth of vertex v_i.
- z_P: Interpolated depth written to the depth map at pixel P.

The face normal is constant over the triangle and is written directly, with no interpolation.

Definition of Feature Maps

Feature maps are the two types of 3D-rendered 2D images defined in Section 2, serving as the direct inputs for SSIM evaluation:

- Normal Map: Stores surface normal direction mapped to RGB color, for light-shadow quality evaluation.
- Depth Map: Stores linear grayscale depth values, for contour and occlusion evaluation.

Each pixel is sampled once, at its centre $(u + 0.5,\ v + 0.5)$: it is covered by the nearest triangle whose projection contains that point and takes that triangle's flat face normal and its perspective-correct interpolated depth; pixels covered by no triangle take the background values below.

Background Pixel Attributes

Fixed background values are assigned to pixels with no triangular mesh intersection:

- Normal Map Background: $\boldsymbol{n}_b = (0, 0, 0)$, mapped to neutral gray $= (127.5,\ 127.5, 127.5)$.
- Depth Map Background: $z_b = 255$ (far plane depth).

Fixed Evaluator Camera Parameters

Participants do not need to use the following camera parameters. However, they are provided to help in designing your algorithm.

Parameter	Symbol	Value	Description
Observation Distance	D	2.5	Distance from camera to mesh origin (model unit)
Focal Length	f_x, f_y	800.0 px	Control projection scaling ratio
Background Depth	z_b	255	Far clipping plane depth value
Background Normal	$\boldsymbol{n}_b$	$(0, 0, 0)$	Background normal vector mapped to neutral gray

Fixed feature map resolution: 1024×1024, principal point $(c_u, c_v) = (512, 512)$ located at the image center.

Constraints

Submitted simplified meshes must satisfy the following constraints. If they do not, you will get `Wrong Answer` and be informed of which one you violated.

Mesh Validity Constraint

- Vertex count: the simplified mesh has $1 \leq V' \leq V$ (at least one vertex, and no more than the original). A submission with $V' = 0$ or $V' > V$ is rejected.
- Manifold mesh: Each edge is shared by exactly two triangular faces (the mesh is a closed, watertight 2-manifold).
- Non-degenerate faces: All triangular faces have positive area.
- Valid indices: All face indices are within vertex array range.

Geometric Deviation Constraint

$$d_H(M, M') \leq 5\% \times Diagonal$$

- $d_H(M, M')$: Symmetric Hausdorff distance between original mesh M and simplified mesh M', defined as $d_H(M, M') = \max\left(\vec{d}(M, M'),\ \vec{d}(M', M)\right)$, where $\vec{d}(A, B) = \max_{a \in A} \min_{b \in B} \|a - b\|$ is the one-way distance from A to B. The first direction requires every original vertex to remain covered by the simplification; the second forbids simplified vertices from straying away from the original surface.
- $Diagonal$: Spatial diagonal length of original mesh axis-aligned bounding box (AABB).

Let original vertex bounds be $x_{min}, y_{min}, z_{min}$ (minimum) and $x_{max}, y_{max}, z_{max}$ (maximum). AABB edge lengths are $L_x = x_{max} - x_{min}, L_y = y_{max} - y_{min}, L_z = z_{max} - z_{min}$, and diagonal length is defined as:

$$Diagonal = \sqrt{L_x^2 + L_y^2 + L_z^2}$$

This normalization limits Hausdorff tolerance to 5% of mesh size, independent of original mesh scale.

Optimization Objective

Subject to the constraints above, participants shall **minimize the vertex count** of the simplified mesh (equivalently, maximize the compression rate defined in the ranking rules). A submission is valid only if its multi-view perceptual score stays at or above the threshold $FinalSSIM \geq 0.9$, where the score is defined as:

$$FinalSSIM = Average_{i=1}^{6} \\ \left(\omega_N \cdot SSIM(I_{N,i}, I'_{N,i}) + \omega_D \cdot SSIM(I_{D,i}, I'_{D,i})\right)$$

- i: View index, total 6 axial views.
- $I_{N,i}, I_{D,i}$: Normal map and depth map of original mesh at view i.
- $I'_{N,i}, I'_{D,i}$: Normal map and depth map of simplified mesh at view i.
- $SSIM(\cdot, \cdot)$: Structural similarity function, output range $[0, 1]$, 1 represents full visual consistency.
- $\omega_N = 0.5$: Weight coefficient of normal map.
- $\omega_D = 0.5$: Weight coefficient of depth map.
- $Average$: Arithmetic mean score of six views.

SSIM Calculation Formula

For two input images X and Y, SSIM is defined as:

$$SSIM(X, Y) = \frac{(2\mu_X\mu_Y + c_1)(2\sigma_{XY} + c_2)}{(\mu_X^2 + \mu_Y^2 + c_1)(\sigma_X^2 + \sigma_Y^2 + c_2)}$$

- μ_X, μ_Y: Mean pixel value inside 11×11 local sliding window.
- σ_X, σ_Y: Local pixel variance of two images.
- σ_{XY}: Local cross-covariance of two images.
- $c_1 = (k_1 \cdot L)^2, c_2 = (k_2 \cdot L)^2$: Stabilization constants, $k_1 = 0.01, k_2 = 0.03, L = 255$ (8-bit pixel dynamic range).
- Final image SSIM is the mean of the per-window SSIM taken over the rendered foreground only. A window is included when the original and/or simplified rendering is non-background at the window's center pixel; windows whose center pixel is the background value (background normal / background depth, defined under "Background Pixel Attributes") in *both* the original and the simplified rendering are excluded. The same foreground-only averaging is applied to each channel of the normal map and to the depth map.

Supplement: For RGB normal maps, SSIM is calculated on three color channels respectively, then averaged for final result.

Evaluation & Ranking Rules

Validity Threshold

A submission is valid only if its total score meets the standard below:

$$FinalSSIM \geq 0.9$$

Submissions below the threshold get 0 points and are excluded from ranking.

Ranking Criteria

The compression rate is defined as:

$$100.0 - 100.0 \times \frac{a_{M'}}{a_M}$$

where $a_{M'}$ is the vertex count of simplified mesh, and a_M is the vertex count of original mesh.

For all valid submissions, ranking priority is determined by simplified vertex count:

- Filter valid meshes with $FinalSSIM \geq 0.9$. If $FinalSSIM < 0.9$, the score of this test case is 0.
- For each test case, the score is the compression rate.
- For all the test cases, the final score is the average of scores.
- Across all test cases, the vertex count is at most $1.1 \cdot 10^6$ and the face count is at most $2.1 \cdot 10^6$. Per-test-case size bounds are listed in the table below.

Test case	$V \leq$	$F \leq$
1 (sample)	10	15
2	5,000	10,000
3	25,000	50,000
4	40,000	80,000
5	50,000	100,000
6	400,000	800,000
7	1,100,000	2,100,000

Input

Input is read using standard input (stdin). The input is a slightly modified version of the OBJ file format.

The first line of input contains the integers V and F ($1 \leq V \leq 1.1 \cdot 10^6$, $1 \leq F \leq 2.1 \cdot 10^6$), the number of vertices and faces of the mesh.

The following V lines each begin with the character `v`, followed by the real numbers x, y, z, the coordinates of a vertex in the mesh. Each coordinate satisfies $-1 \le x, y, z \le 1$ and is given with at most 15 digits after the decimal point. The mesh is **pre-normalized**: its axis-aligned bounding box is centered at the origin (so $x_{\min} = -x_{\max}$, and likewise for y and z) and every vertex lies within the unit sphere, $\sqrt{x^2 + y^2 + z^2} \le 1$.

The final F lines each begin with the character `f`, followed by the integers v_1, v_2, v_3 ($1 \le v_1, v_2, v_3 \le V$), meaning that there is a triangular face connecting the vertices numbered v_1, v_2 and v_3.

The input mesh is guaranteed to be a closed, watertight 2-manifold: every edge is shared by exactly two faces and the surface is connected. Every face is non-degenerate (its three vertices are distinct and span a positive area), and there are no duplicate vertices or duplicate faces.

Check the sample input file below for a precise example of the format.

Output

Write your simplified polygon to standard output (stdout) in the same format as the input. The output mesh must be manifold. Additionally, it may not have any zero-area degenerate triangular faces.

The output may be at most 100 MiB in total. Thus, you should take care not to print an excessive amount of decimals if your mesh is large. The baseline solutions described below respect this bound.

Writing Your Solution

Because this problem has a large amount of input, we have provided C++ and Python code that can quickly read and write input/output. They are provided in the attachments section at the bottom of the page. These are provided as-is: you may modify them in any way you see fit, and you may of course choose not to use them.

Additionally, for C++, the library Eigen is available. The Eigen files will be placed in the same directory as your solution when compiled, and can be included using for example `#include "Eigen/Dense"`. The version provided is 5.0.0. You do not need to submit any Eigen files. It might be beneficial to download Eigen for local debugging and development of your solution if you choose to use it.

Explanation of Sample

In the sample shown below, the simplified mesh removes a redundant vertex from the right face of original mesh without changing the face normal and depth. Therefore, $FinalSSIM = 1$, and the compression rate is $100.0 - 100.0 \times \frac{8}{9} \approx 11.11$. You are not awarded any points for solving the sample case, but can use it to debug your solution.

Sample Input 1

```
9 14
v 0.5 0.5 0.5
v 0.5 0.5 -0.5
v 0.5 -0.5 0.5
v 0.5 -0.5 -0.5
v -0.5 0.5 0.5
v -0.5 0.5 -0.5
v -0.5 -0.5 0.5
v -0.5 -0.5 -0.5
v 0.5 0.49 0.49
f 1 3 9
f 1 9 2
f 9 3 4
f 9 4 2
f 5 6 8
f 5 8 7
f 1 2 6
f 1 6 5
f 3 7 8
f 3 8 4
f 1 5 7
f 1 7 3
f 2 4 8
f 2 8 6
```

Sample Output 1

```
8 12
v 0.5 0.5 0.5
v 0.5 0.5 -0.5
v 0.5 -0.5 0.5
v 0.5 -0.5 -0.5
v -0.5 0.5 0.5
v -0.5 0.5 -0.5
v -0.5 -0.5 0.5
v -0.5 -0.5 -0.5
f 1 3 4
f 1 4 2
f 5 6 8
f 5 8 7
f 1 2 6
f 1 6 5
f 3 7 8
f 3 8 4
f 1 5 7
f 1 7 3
f 2 4 8
f 2 8 6
```